

UML

Gregorics Tibor and Pintér Balázs

Unified Modeling Language (UML)

- ❑ UML (Unified Modeling Language) is a graphical language used to document the phases of the object-oriented software development process, including analysis, design, implementation, and testing.
- ❑ The diagrams can be used to present different parts of a software system, either under development or already completed, from various viewpoints. They are used to plan and document the system.
- ❑ The diagrams are grouped according to whether they describe the structure or behavior of a system
 - **structural view diagrams**: object, class, component, deployment, package diagram
 - **behavioral view diagrams**: use case, communication, sequence, state machine, activity diagram

The diagrams used in the course

- **use case diagram** (analysis)
 - the main features or user stories
- **object diagram** (analysis)
 - the objects and their relationships existing at a given point in time
- **communication diagram** (analysis)
 - message exchanges between objects (like method calls)
- **class diagram** (analysis, planning, implementation)
 - objects (classes) and the relationships between them
- **sequence diagram** (analysis, testing)
 - a possible scenario of message exchanges between objects on a timeline
 - there are usually multiple sequence diagrams for different scenarios
- **state machine diagram** (planning)
 - illustrates the state changes of an object (its operation) in response to the messages sent to it, such as calling one of its methods

UML

1st part

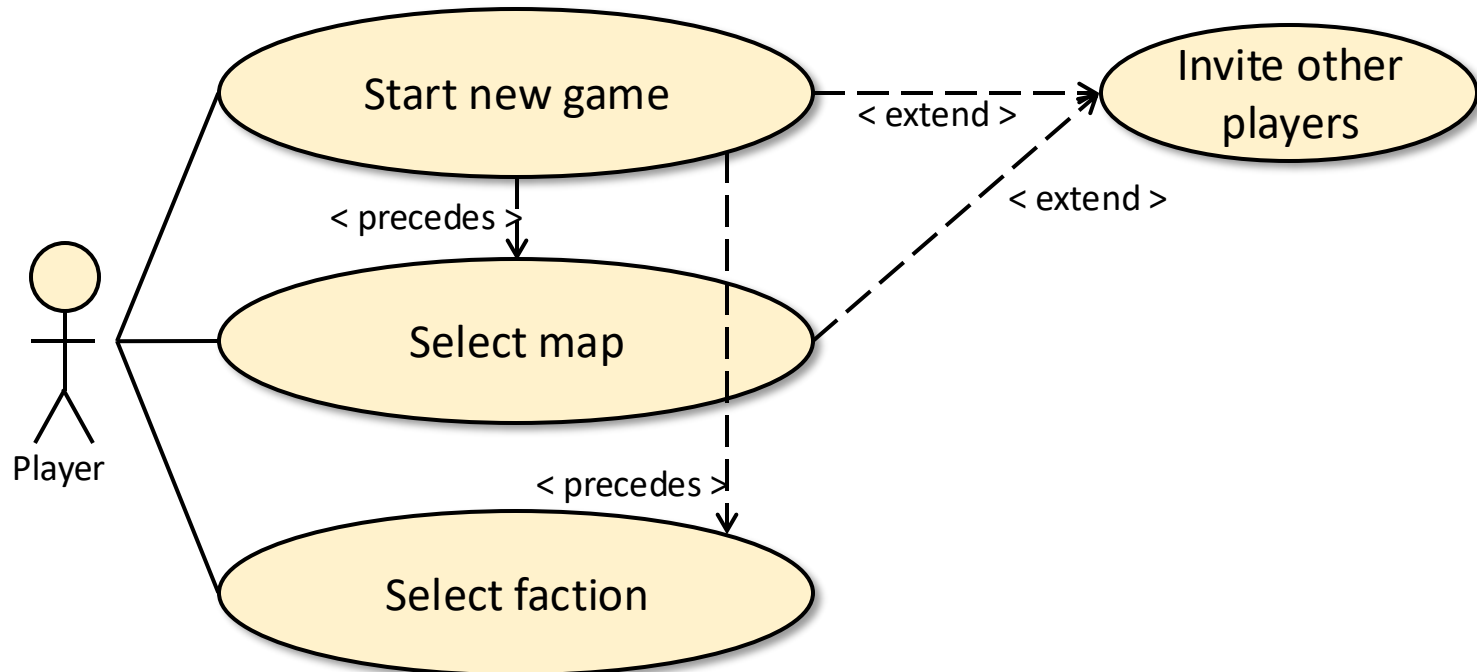
Use case diagram

Gregorics Tibor and Pintér Balázs

Use case diagram

■ Shows about the planned system

- what functions does it perform, what services does it provide
- for whom it provides services
- what kind of expectations does it have of its environment
- what kinds of relationships should exist between individual features



Elements of a use case diagram

- ❑ **Use cases:** activities (functions, services) that can be identified and initiated from the outsiders perspective (i.e., the user or other external parties, actors) of the system
- ❑ **Connections** among the use cases that
 - depict their breakdown into parts or its supplementation with additional functions
 - determine their order
 - point to their derivation from a general use case
- ❑ **Actors:** users or other external entities that use the system's functions and drive its intended operation. An actor can be designated as belonging to a specific type of actor (specialization). The number of instances of a particular type of actor can also be specified (multiplicity).

Use case relationships

□ Parts of a use case

- **include**: points to a well-delineated, independently initiatable part of a use case without which the containing activity would no longer be complete but rather abstract.
- **extend**: refers to another case that optionally complements a complete use case (which is not abstract)

□ Order of use cases

- **precedes**: designates the order of activities that can be initiated directly by the actor.
- **invokes**: indicates which other activity must follow the execution of an activity regardless of the actor's will

□ Derivation between use cases

- a generally stated use case has multiple distinct concrete variations

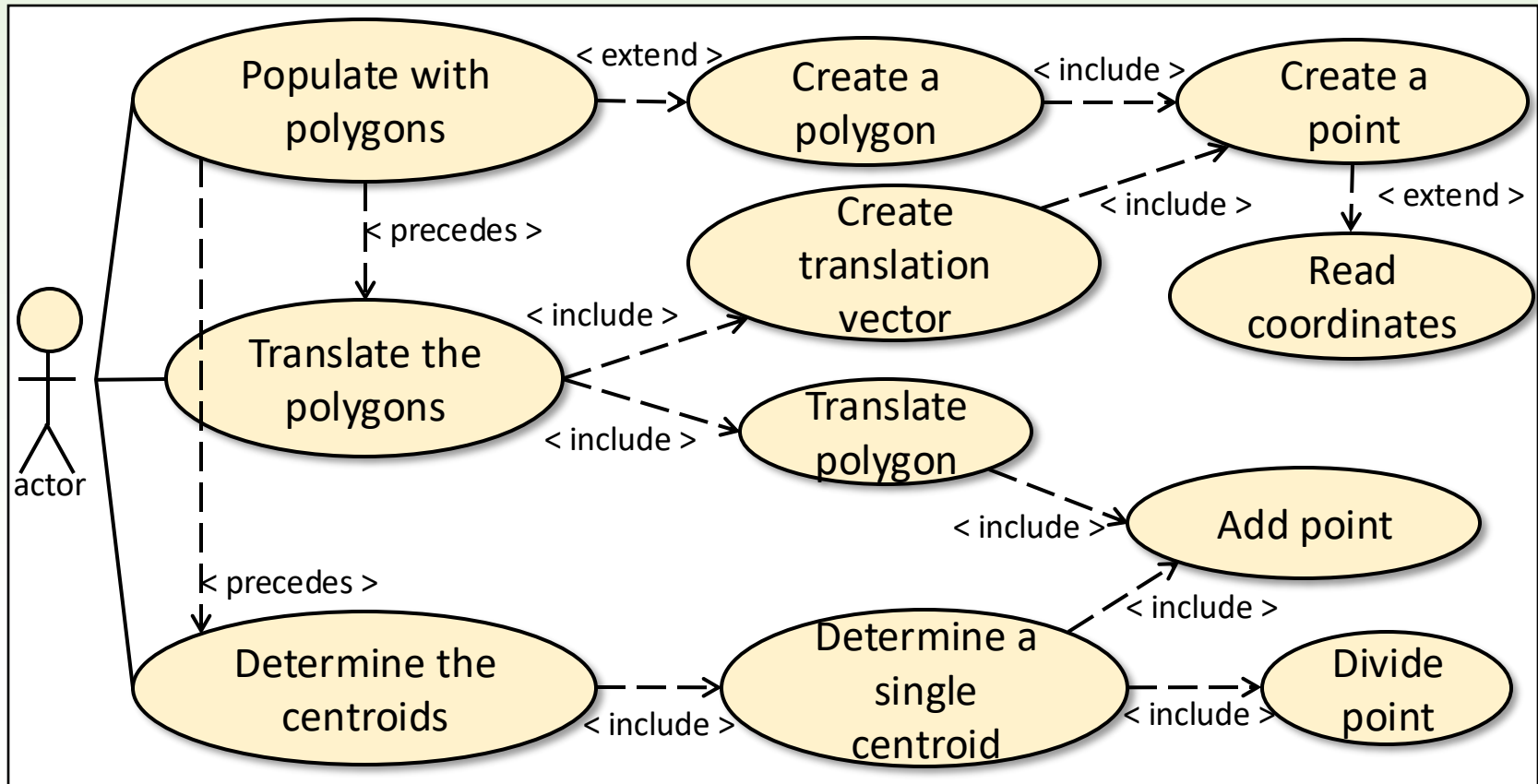
User story

- ❑ The use case diagram by itself does not provide a sufficient picture of the functionality of the system, as it essentially only gives the name of the individual activities.
- ❑ The use cases can be detailed in a tabular description of the user stories:
 - **name** of the activity
 - **precondition** of the activity (GIVEN)
 - what kind of **event** makes the activity happen (WHEN)
 - the **effect** or result of the activity (THEN)
 - which kind of users it is provided for

AS a kind of user		
Use case		Description
Name of activity	GIVEN	Precondition
	WHEN	Event that makes the activity happen
	THEN	Effect that happens

Example

Let's fill a container with various polygons. Translate each of them in the same direction and by the same amount, then calculate the centroids of the resulting polygons. The coordinates of the vertices, centroids, and the endpoints of the displacement vectors are all real numbers.



Example

Case		Description
Populating the container with polygons.	GIVEN	A container (like a sequence) that can store polygons.
	WHEN	Request to place a polygon in the container.
	THEN	The container holds polygons.
Creating a polygon.	GIVEN	The application is waiting to create a polygon.
	WHEN	Specifying the vertices (their number and the points).
	THEN	a polygon (object) is created
Creating a point	GIVEN	Coordinates of a point on a plane
	WHEN	Creating a point in the plane.
	THEN	a planar point (object) is created
Translating a container's polygons.	GIVEN	Container for polygons.
	WHEN	Translating polygons by specifying the displacement vector.
	THEN	The container shifts its polygons one after the other by the given vector.
Translating a point	GIVEN	Coordinates of a point in the plane
	WHEN	Giving the displacement vector
	THEN	Creating a new, translated point in the plane.

Example

Case		Description
Determining the centroids of the polygons	GIVEN	container with polygons
	WHEN	Request to determine centroids.
	THEN	The container calculates the centroid of each polygon in sequence.
Calculating a polygon's centroid.	GIVEN	Polygon
	WHEN	Request to calculate the centroid
	THEN	Based on the vertices of the polygon, a planar point (object) is created.
Translate polygon	GIVEN	the polygon and the endpoint of the displacement vector
	WHEN	Request to translate
	THEN	The polygon's vertices have been translated.
Translate point	GIVEN	Point and displacement vector
	WHEN	Request to translate
	THEN	We add the coordinates of the displacement vector to the coordinates of the given point.
Divide point by scalar	GIVEN	a point in the plane and an scalar integer
	WHEN	Request to divide
	THEN	The coordinates of the point are divided by the integer

UML

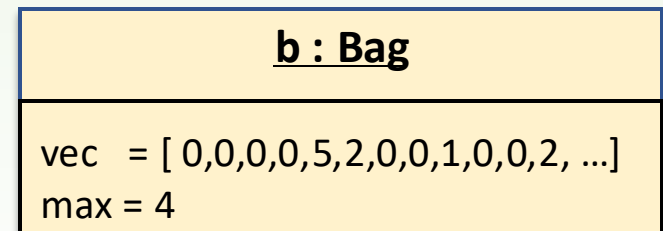
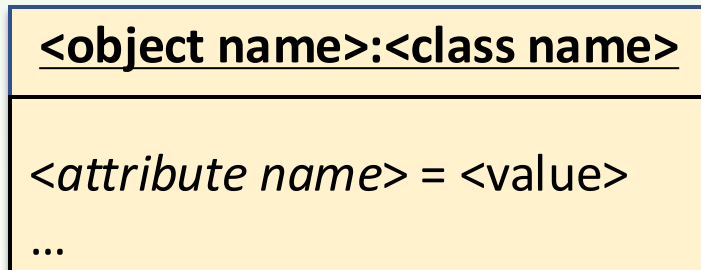
2nd part

Object and communication diagram

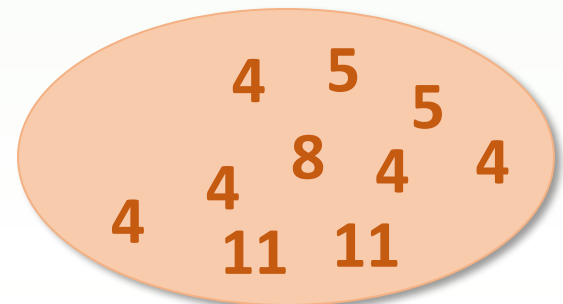
Gregorics Tibor and Pintér Balázs

Object diagram

- ❑ The object diagram is used to describe the objects that exist at a particular point in time, which are collectively the **population**.
- ❑ An object is determined by
 - its **class**, which characterizes objects that have the same data attributes and methods
 - its **name** (which is not mandatory)
 - its **state** (determined by the values of its attributes)

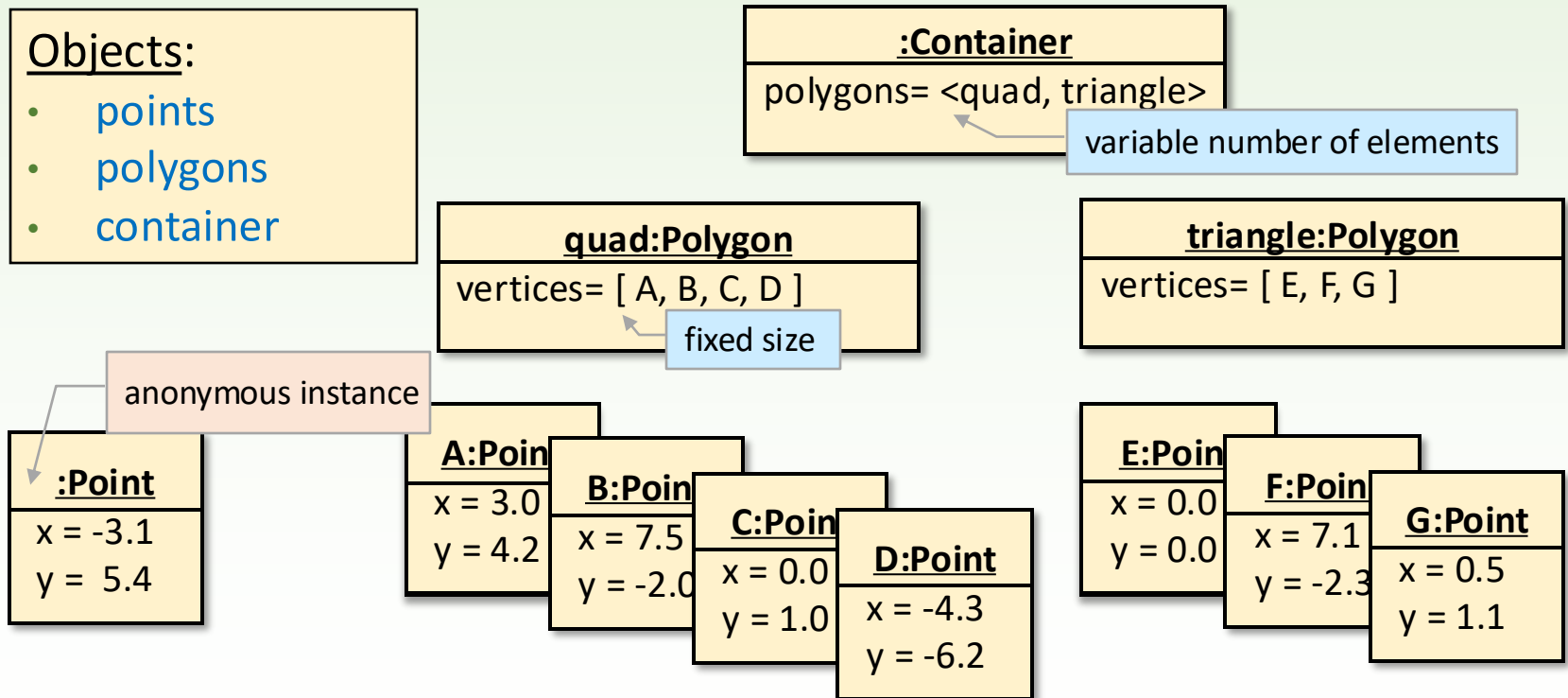


The object diagram does not include the methods that can be invoked on the object.



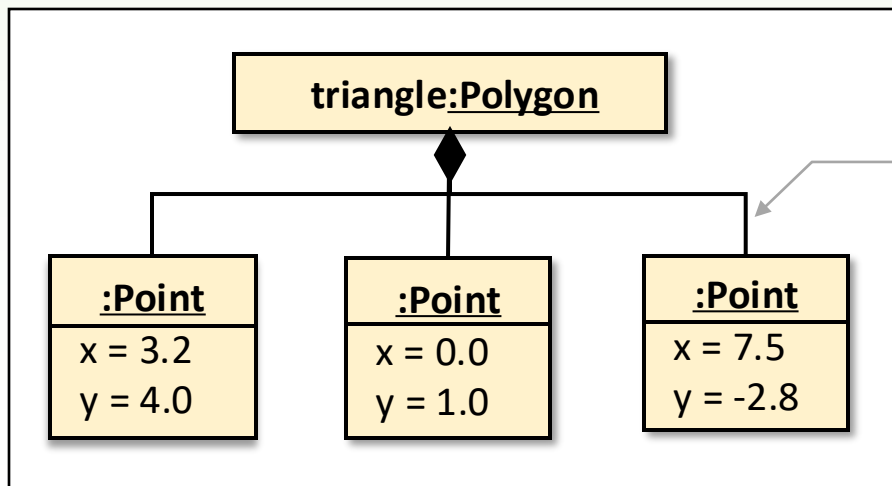
Example

Let's fill a container with various polygons. Translate each of them in the same direction and by the same amount, then calculate the centroids of the resulting polygons. The coordinates of the vertices, centroids, and the endpoints of the displacement vectors are all real numbers.



Object relationships

- ❑ Relationships can form between objects for various purposes:
 - an object may reference another object, like one of its components
 - an object's method can call another object's method,
 - an object can send a signal to another object
- ❑ In a relationship one object must know the reference to the other object: it can store this in one of its data attributes, one of its methods can take it as a parameter, or it may create it itself.



A continuous line indicates a relationship between two objects. It may be terminated by an arrow pointing to the object that is accessible from the other, or a rhombus **on the other end** if the object indicated contains the objects related to it.

Responsibility of objects

- ❑ A method of an object performs an activity with the object's data attributes that is responsible for a part of solving the task.
- ❑ When we assign the activities necessary to solve a task as methods to individual objects, we thereby designate the scope of responsibility the object has in solving the task.
- ❑ We need to make sure to distribute these areas of responsibility proportionally among the objects.

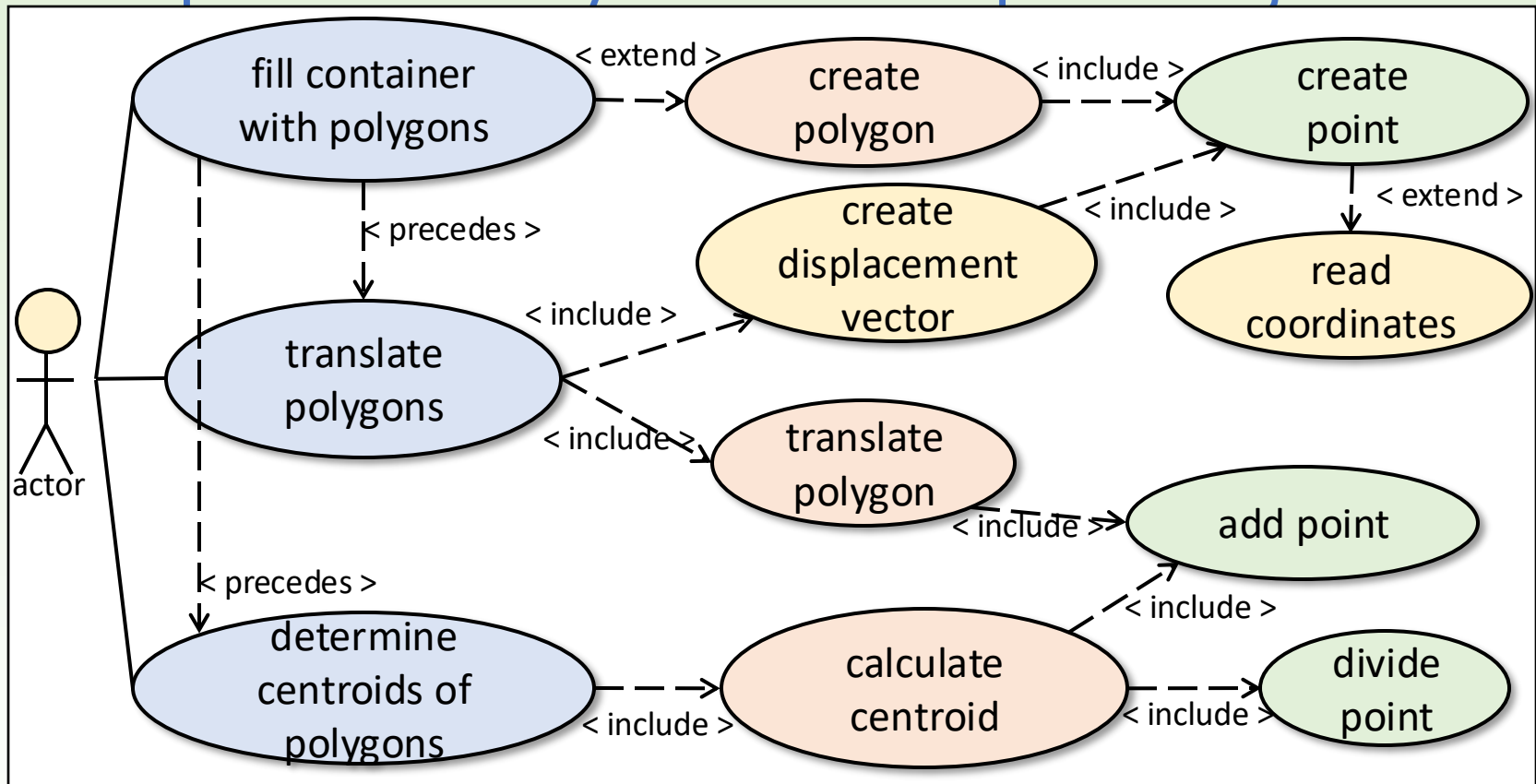
Single responsibility

O
L
I
D

One object – one responsibility

- There should not be "all-powerful" objects that also have methods affecting multiple spheres of responsibility.
- Multiple objects shouldn't share the same scope of responsibility.

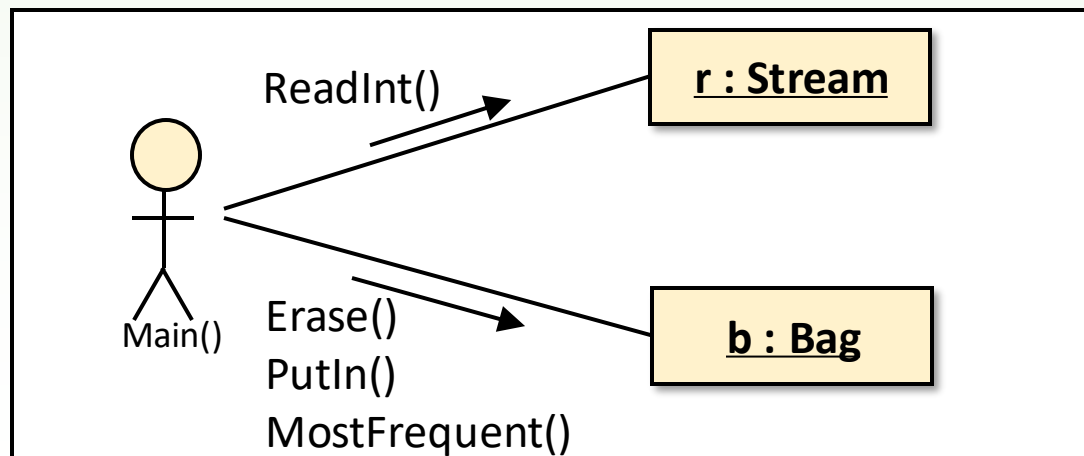
Responsibility of example objects



- point: create, translate, divide
- polygon: create, translate, compute centroid
- container: add polygon, enumerate polygons

Communication diagram

- ❑ This diagram resembles an object diagram but does not show the value of the objects' attributes, instead it shows what **messages** the related objects can **exchange** with each other.
- ❑ A message can be a **method invocation** or sending a **signal**.
- ❑ An arrow drawn next to a connection pointing from the **sender** object to the **receiver** object indicates the direction of the connection, which can be two-way. We write the methods/signals owned/managed by the receiver object on the arrow, thus the diagram essentially designates the scope of responsibility of each object, including its methods, and compensates for the lack of this information in the object diagram.

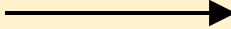


Messages of a communication diagram

□ Messaging is, when

- the sender object invokes one of the **methods** of the receiver,
- the sender object sends a **signal** to the receiver object.

□ The way of messaging can be

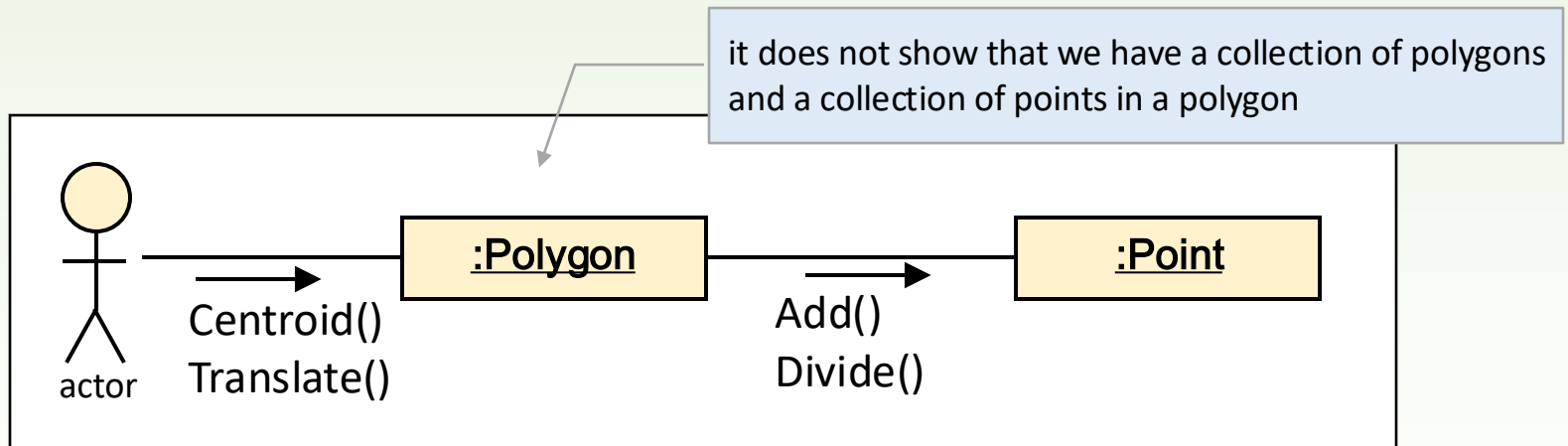
- **synchronous**: The sending object waits until the recipient processes its message and only after receiving the possible result continues its own activity, 
- **asynchronous**: After sending the message, the activities of the sender and receiver objects run in parallel. 

□ With additional notation

- The **order** of messages can be designated by numbering them.
- The message **precondition** can be specified in square brackets.
- It can be flagged with a star if a message is **repeating** in a cycle.

Example

Let's fill a container with various polygons, push each of them in the same direction and by the same amount, and then calculate the centroids of the resulting polygons. The coordinates of the vertices and centroids and the coordinates of the endpoint of the displacement vector are integers.



UML

3rd part

Class diagram

Gregorics Tibor and Pintér Balázs

Class diagram

- ❑ A class is able to generally define the data attributes and methods of objects of the same type.
- ❑ To describe different types of objects, a diagram containing multiple classes must be created:
 - a general description of the object population
 - the relationships between the (classes of) objects
- ❑ One class diagram can correspond to many object diagrams, in other words, the class diagram is an abstraction of object diagrams.
- ❑ A class diagram is primarily a tool to support software design, but it can also be used for software analysis, as well as for describing the implementation.

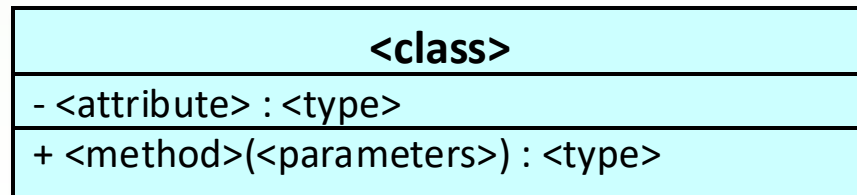
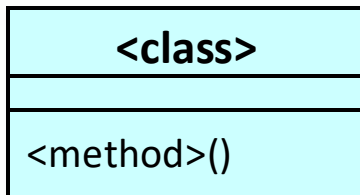
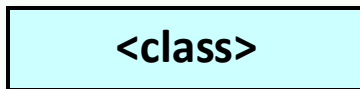
Elements of a class

- ❑ The description of a class characterizes the objects that can be instantiated from it. It contains the:
 - **name of the class**, which is also the name of the type of the objects
 - **attributes** (data members): their name, type, and other features
 - **methods** (operations, member functions): their name, parameters, return type, and other features
- ❑ The visibility of attributes and methods shows whether we can access the member from outside the class or not.
 - **public (+)**: visible
 - **private (-)** and **protected (#)**: hidden

<name>
<+ - #> <attribute name> : <type>
...
<+ - #> <method name>(<parameters>) : <type>
...

How detailed is the class

- ❑ The class diagram **develops during modeling**: it is initially incomplete, then expanding gradually in multiple steps, possibly up to where it contains all the necessary information for implementation. A possible workflow is:
- the names of the main classes are created first, followed by the names of the methods that cover the areas of responsibility.
 - we include the data members with their names and types.
 - we decide visibility of attributes
 - we refine the **signature** of the methods and possibly give their body
 - lastly we give the hidden helper methods



Example

Let's fill a container with various polygons. Translate each of them in the same direction and by the same amount, then calculate the centroids of the resulting polygons. The coordinates of the vertices, centroids, and the endpoints of the displacement vectors are all real numbers.

<u>quad:Polygon</u>
vertices= [A, B, C, D]

<u>triangle:Polygon</u>
vertices= [E, F, G]

Polygon
vertices: Point[]
centroid() translate()

<u>A:Point</u>
x = 3.0 y = 4.2

<u>B:Point</u>
x = 7.5 y = -2.0

<u>C:Point</u>
x = 0.0 y = 1.0

<u>D:Point</u>
x = -4.3 y = -6.2

<u>E:Point</u>
x = 0.0 y = 0.0

<u>F:Point</u>
x = 7.1 y = -2.3

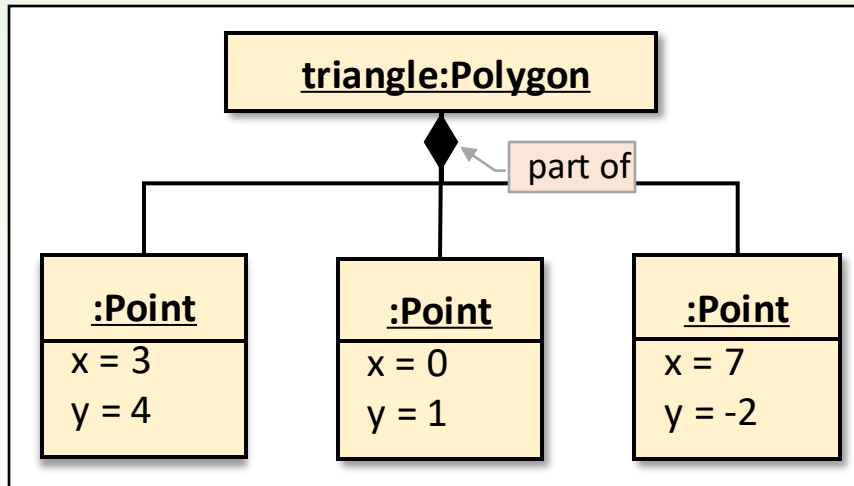
<u>G:Point</u>
x = 0.5 y = 1.1

Point
x : real y : real
add() divide()

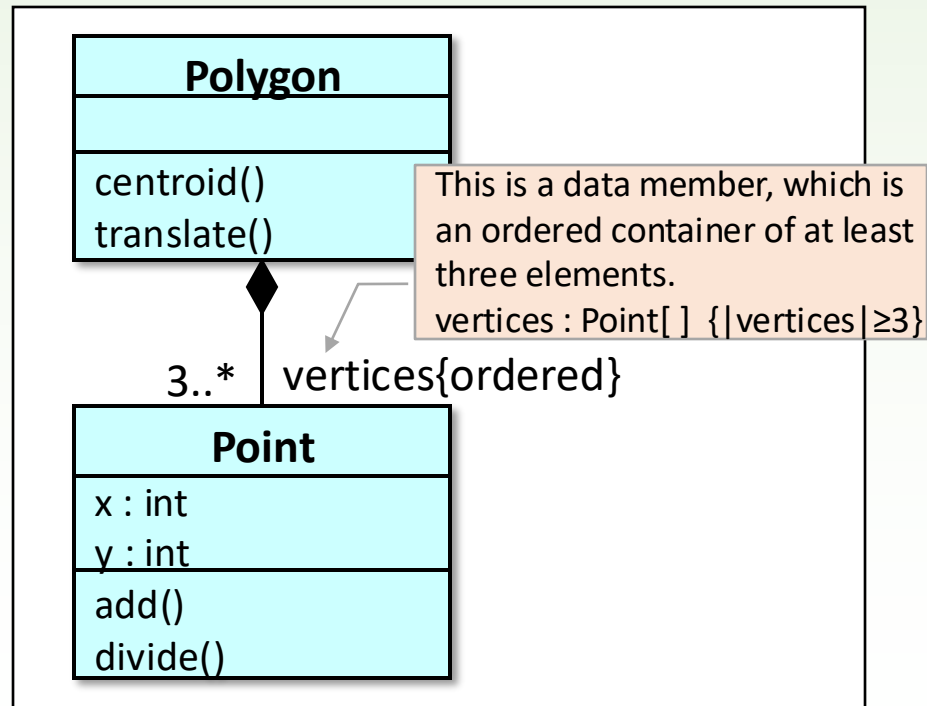
Relationships between objects on a class diagram

- We will discuss the class-level description of relationships between objects in more detail in the next lecture.

Representation of relationships between objects in an object diagram

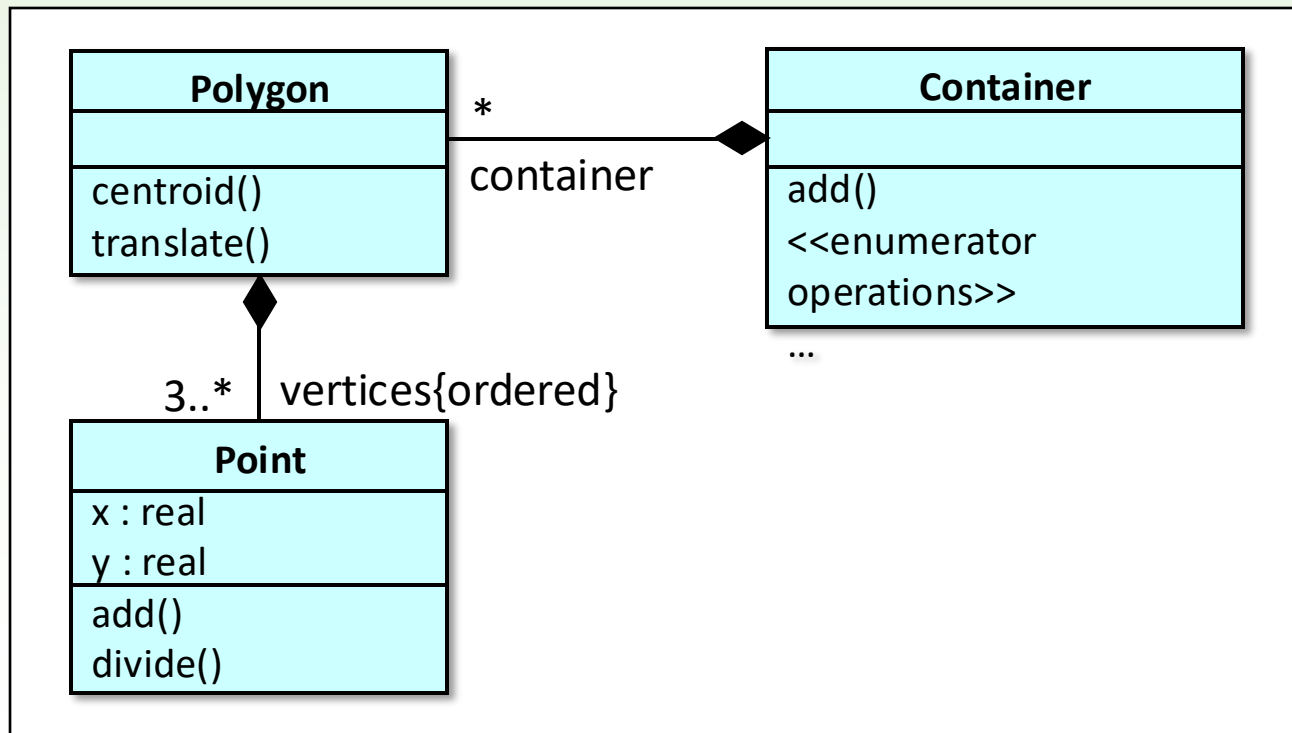


Representation of relationships between objects in a class diagram



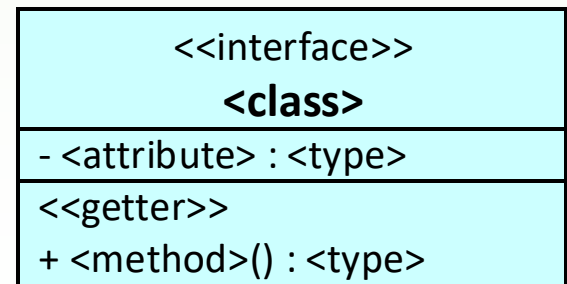
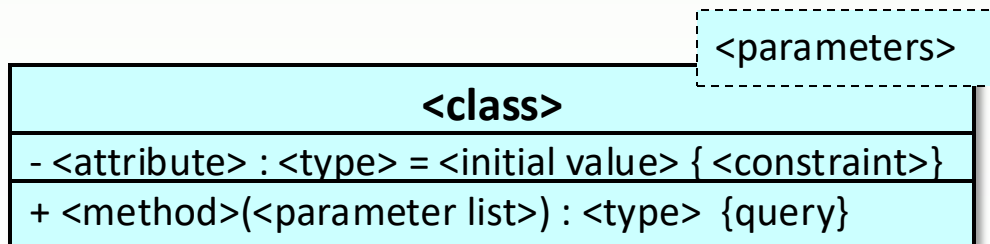
Example

Let's fill a container with various polygons. Translate each of them in the same direction and by the same amount, then calculate the centroids of the resulting polygons. The coordinates of the vertices, centroids, and the endpoints of the displacement vectors are all real numbers.



Class diagram extensions

- ❑ Classes can be categorized into various **stereotypes** based on the characteristic behavior of their instances, which can be described within <<...>> notation (e.g., <<interface>>, <<enumeration>>, <<singleton>>).
- ❑ A class can also be written in a **generalized** way (as a generic or template class) if we introduce parameters (e.g., type placeholders).
- ❑ You can assign **initial values** to the data fields (after the '=' sign), which the constructor will set, and you can also specify **constraints** (type invariants) for them using the {...} notation.
- ❑ **Properties of methods** (e.g., query, virtual, override) can also be specified within {...} notation. The <<getter>> labels a group of methods that **retrieve** the values of data members, while the <<setter>> introduces operations that **overwrite** the values of data members.



UML

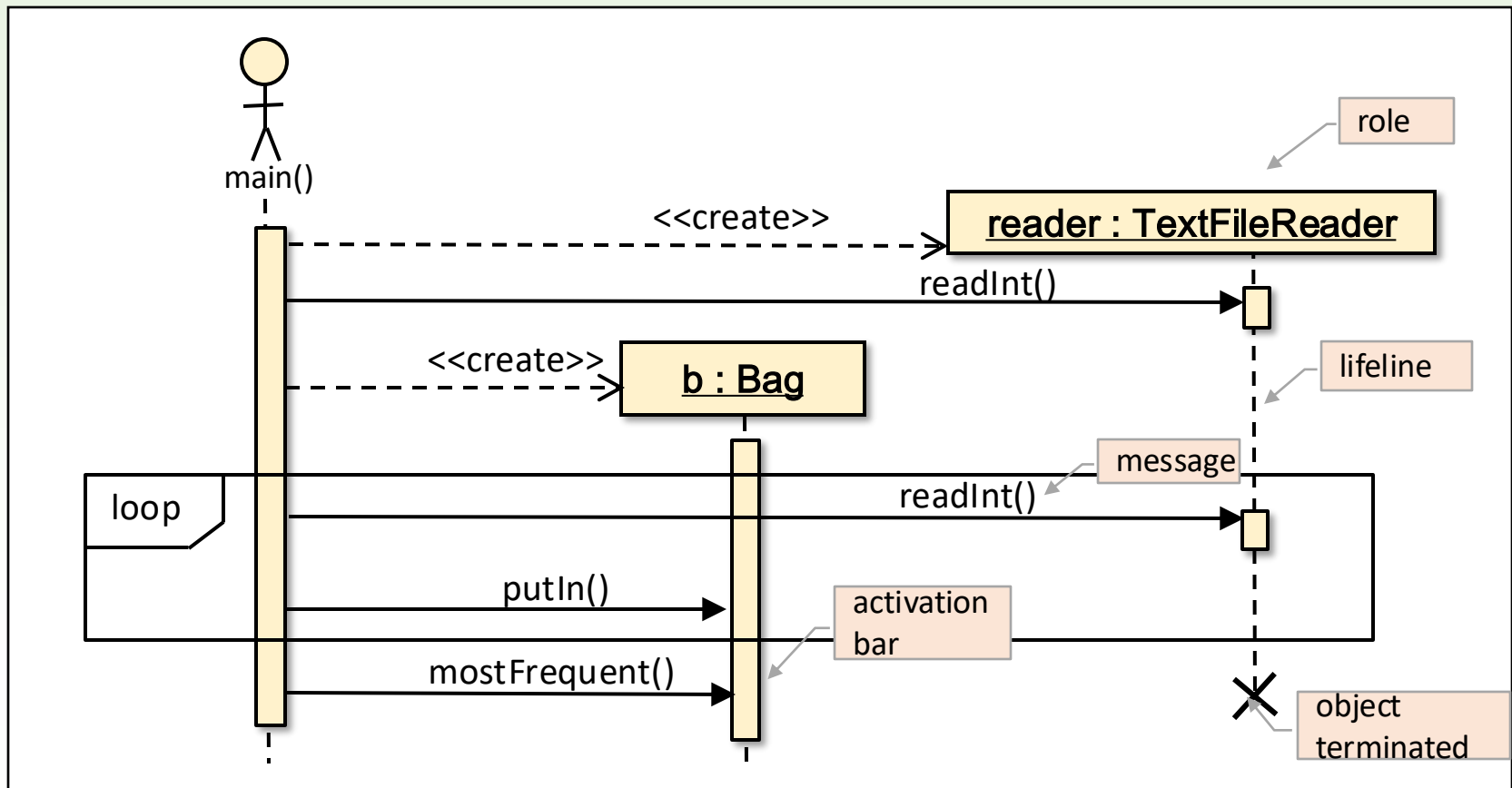
4th part

Sequence diagram

Gregorics Tibor and Pintér Balázs

Sequence diagram

- Shows the **chronological order** of message exchanges between objects.
- Describes a possible operation of the system or a part of the system
Usually, multiple diagrams are created, for example, for the use cases. It is used not only for analysis but also for documenting testing.

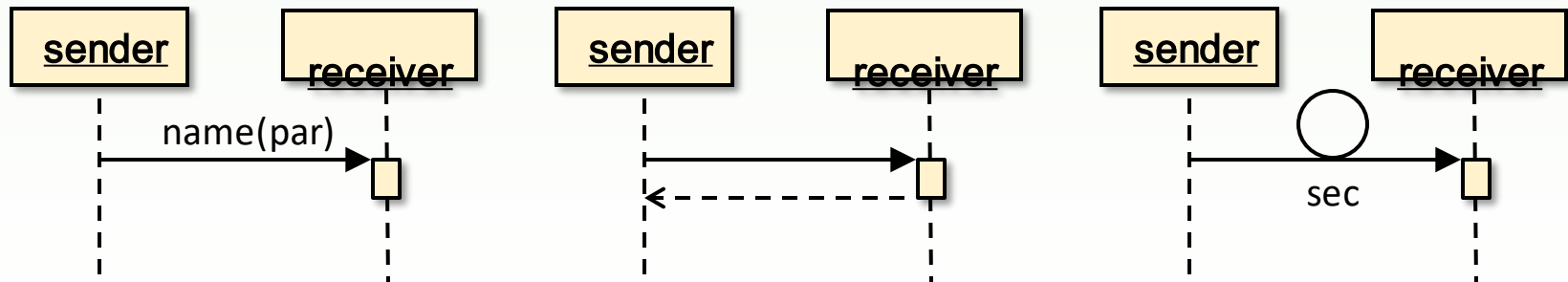


Elements of a sequence diagram

- ❑ **Role** (top of the diagram) : It embodies one or more objects of one or even multiple classes, often just a single object.
- ❑ **Lifeline** (dashed vertical line starting from the role): a timeline depicting the existence of a role.
- ❑ **Activation bar** (a vertically stretched rectangle on the lifeline) : A period of a lifeline during which the objects embodying the role are under the control of other objects as their methods are being executed.
- ❑ **Message** (usually a horizontal arrow between the lifelines) : The form of information transfer between objects. It shows how one object activates (triggers an action of) another object by sending a message (e.g., **invoking one of its methods**).
 - The messages below each other show their chronological order.
 - The elapsed time between two messages can also be displayed.

Synchronous messages.

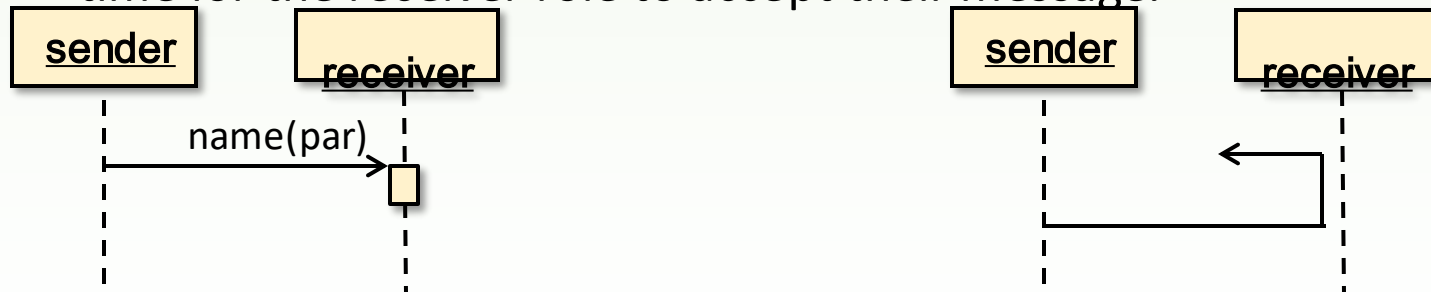
- ❑ The sender role (object) transfers control to the receiver role (object) and blocks its own activity until the receiver releases control. This can be...
 - The synchronous **call of a method** of the receiver or (rarely) the synchronous sending of a signal to it. The message tag is the name of the called method/signal with any potential parameters.
 - A **synchronization message** during which the sender role waits for feedback from the receiver role.
 - A time-bound **waiting message**, when the sender role waits for a designated period for feedback from the receiver role.



Asynchronous messages



- ❑ The sender **does not interrupt its own activity** after sending the message and does not wait for feedback from the receiver role. This can be
 - An **asynchronous method call** or an **asynchronous signal**, after which the sender and receiver roles (objects) operate in parallel. The message label is the name of the invoked method/signal with the parameters.
 - A **rendezvous message**, where the sender role waits for a period of time for the receiver role to accept their message.



Special messages

----->

- ❑ These are the messages that we often don't even display for the sake of easier readability.
 - A **return message** sent by the receiving role to the sender role in response to a previously received synchronous message, in order to release its blocking. In the case of a synchronous method call, this is self-explanatory (and therefore often omitted), but the response to a synchronous message must always be explicitly indicated.
 - Instance **creation** message: creates a new role (object).

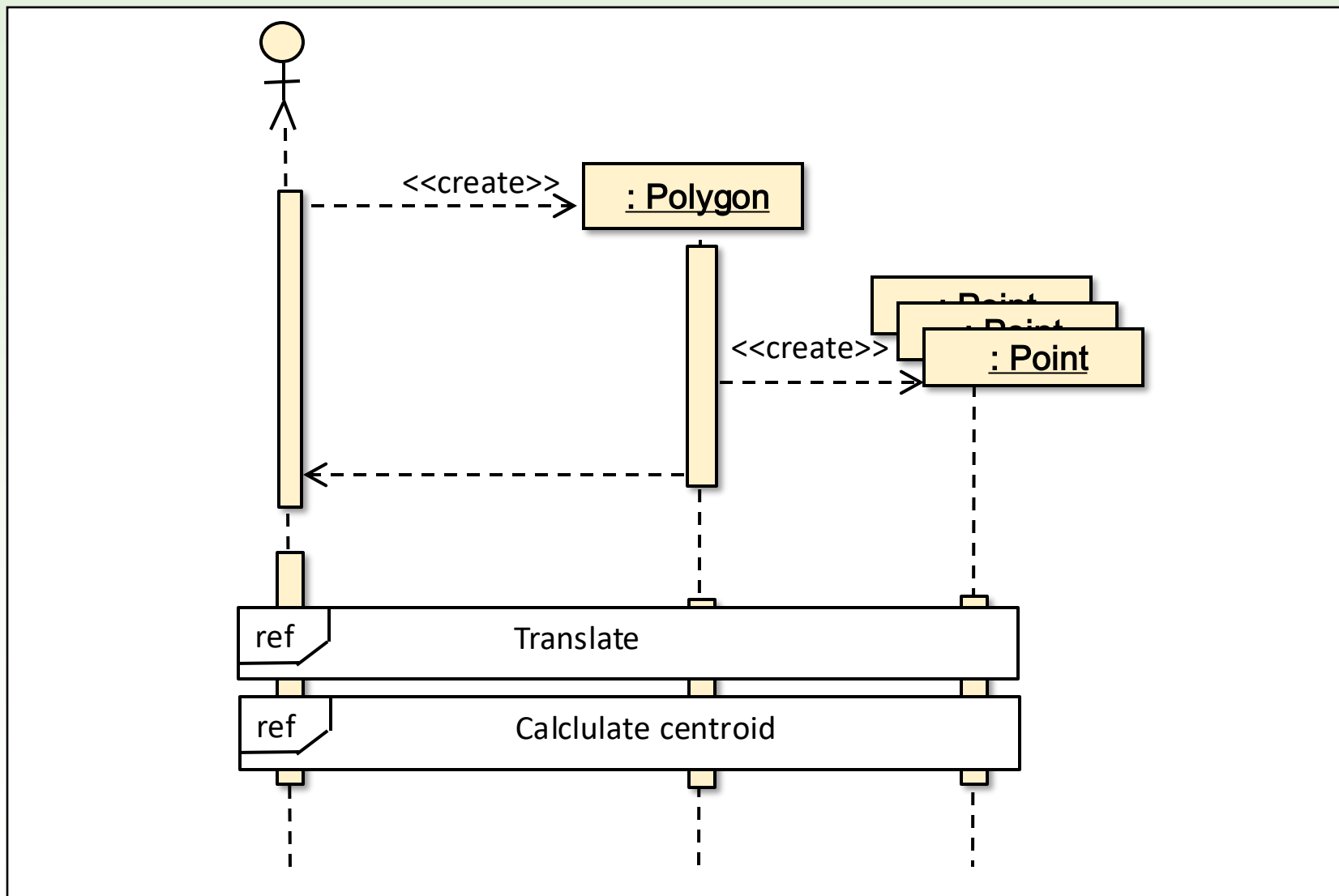


Hierarchy of sequence diagrams.

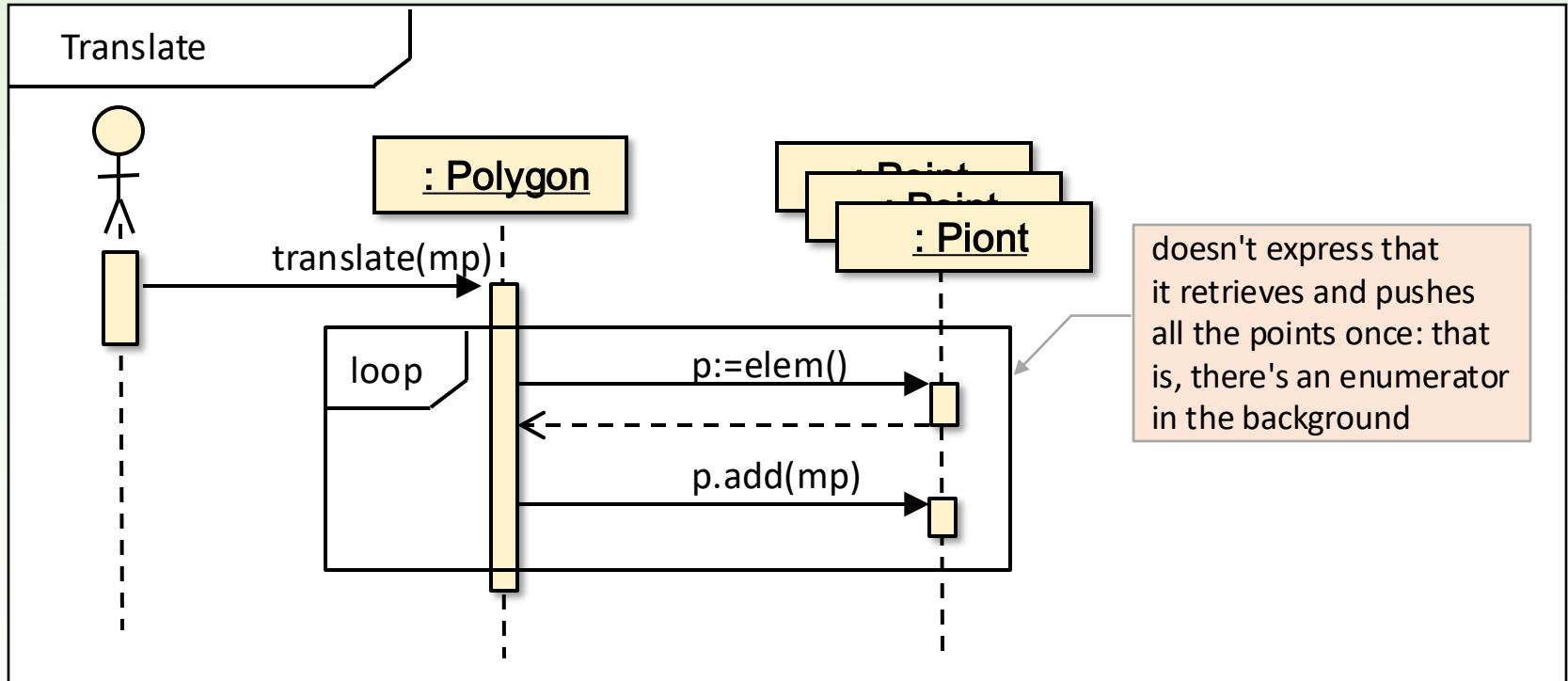
There is an option to hide certain parts of a complex sequence diagram for the sake of readability, and these can later be detailed and refined separately.

- ❑ A sequence diagram can hide, for example, the message exchanges between the objects or groups of objects that make up a single role.
- ❑ A specific portion of a given time interval in the diagram can also be separated (ref).
- ❑ We can highlight certain details (fragments) that
 - are looping (loop)
 - will only occur under certain conditions. (alt, opt)
 - are running in parallel. (par)
 - ...

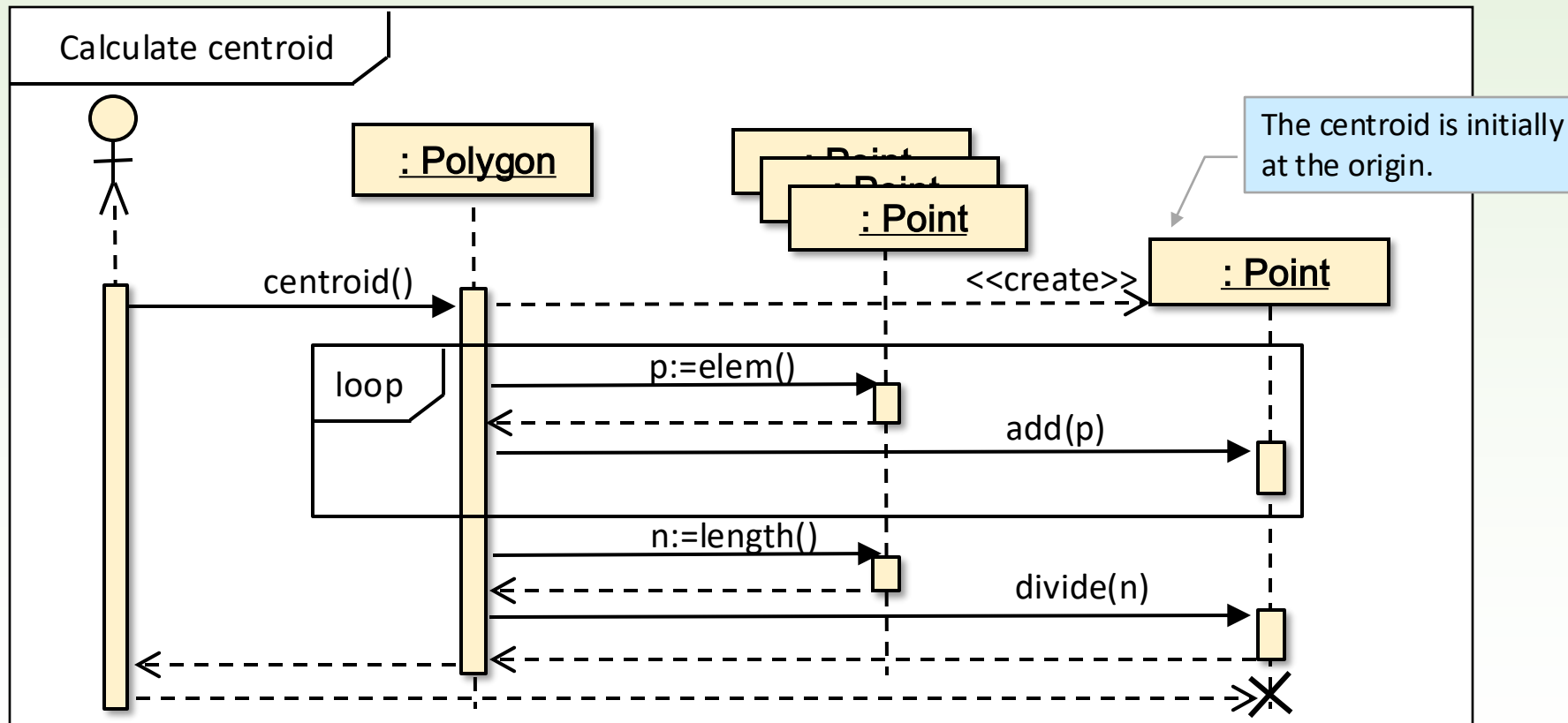
Example



Example



Example



UML

5th part

State machine diagram

Gregorics Tibor and Pintér Balázs

State machine diagram

- ❑ An object can have various states. There are two kinds of states:
 - the values of all the attributes all together (**physical state**)
 - subsets of physical states, like not empty array (**logical state**)
- ❑ Among the **states** of the object, only one can be **active** at a time, which may change in response to an **event**: a **state transition**.
- ❑ An event can be a **message** sent to an object (calling the object's method or sending a signal to the object) or the occurrence of a condition.
- ❑ The changes in the state of the object can be represented by a **directed graph**, in which
 - The nodes indicate the states of the object.
 - The directed edges show the transitions between states, and the labels on the edges indicate the event that triggers the transition.
- ❑ **We'll discuss this diagram in much greater detail later on.**

State machine of enumerator

- ❑ We distinguish three states of an enumerator object.: **ready to start, processing, finished**.
- ❑ The operations of enumeration are only meaningful in certain states; otherwise, their effect is undefined.

